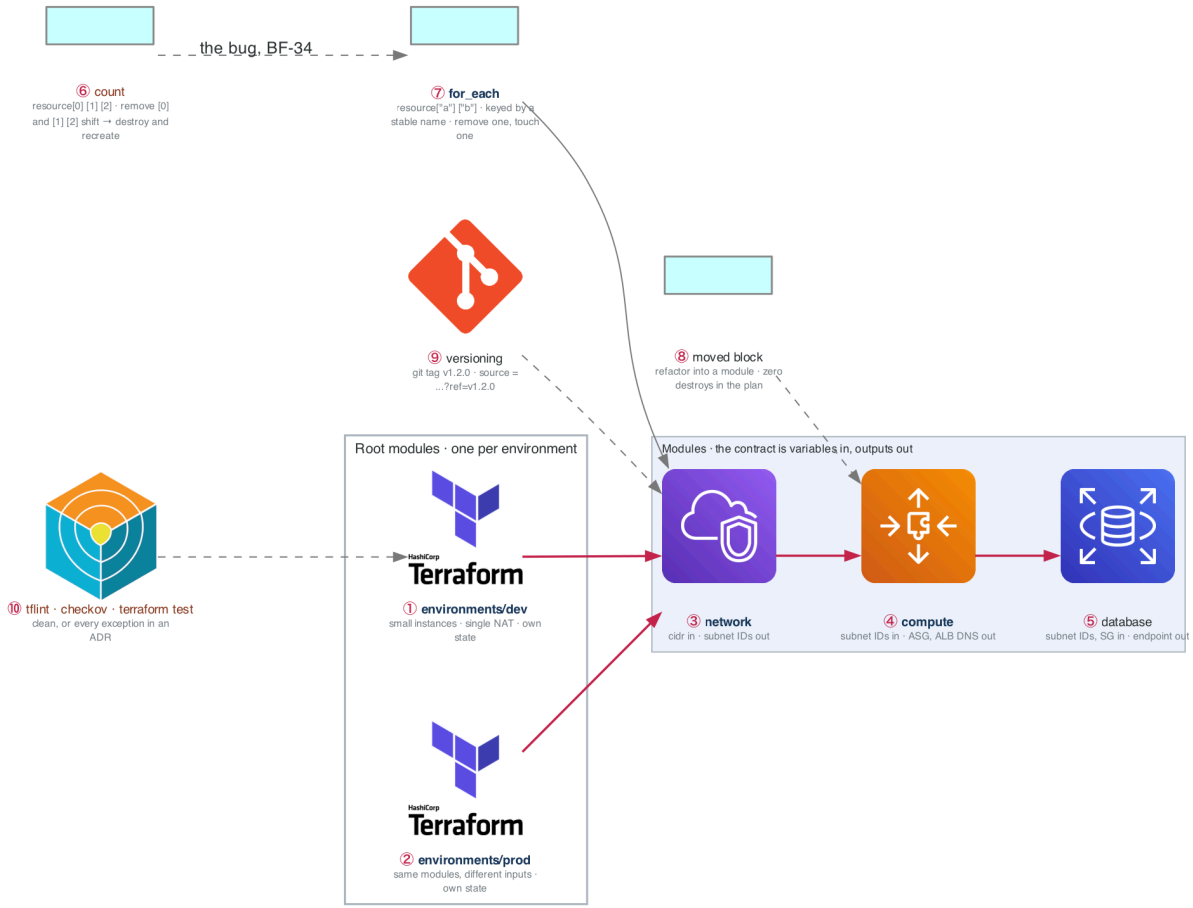
Module contract, environments, and the count index shift, numbered

Week 11 · Figure 11.1 · figure with badges 1–10 · 2026-10-17



What to say, badge by badge

About three and a half minutes at a normal pace (376 words). Point at each badge as you say its line.

Open
“Anyone can write a main.tf. This week is about the shape of code a second engineer can consume, two environments that share it, and the one iteration mistake that deletes production subnets. The orange box is that mistake.”

1 environments/dev
“A root module with its own state file and small, cheap inputs: one NAT, small instances. It calls the same three modules prod does.”

2 environments/prod
“Same modules, different variable values, separate state. The test that you got this right: a change to a module shows up in both plans, and a change to an input shows up in one.”

3 network module
“Takes a CIDR, returns subnet IDs. The caller never sees route tables or NAT gateways. That is the contract: variables in, outputs out, implementation private.”

4 compute module
“Takes subnet IDs, returns the ASG and the ALB's DNS name. It knows nothing about how the subnets were made. Modules compose through outputs, never by reaching into each other.”

5 database module
“Takes subnet IDs and a security group, returns an endpoint. The password is generated by RDS and lands in Secrets Manager; Terraform never holds it.”

6 count
“Three subnets as resource[0], [1], [2]. Remove the first and Terraform sees [1] become [0] and [2] become [1]: it plans to modify two and destroy one. In production that is three subnets gone to remove one. BF-34, and you reproduce it before you fix it.”

7 for_each
“The same three subnets keyed by a stable name: resource["a"], ["b"], ["c"]. Remove "a" and the plan touches exactly one. The migration from count to for_each is a set of state mv commands, which you also do by hand.”

8 moved block
“Pull a resource into a module and the address changes. A moved block says old address, new address, and the plan shows zero destroys. The exit criterion is that zero.”

9 versioning
“Tag the module repository and pin the ref in the source string. An environment on v1.2.0 does not change when v1.3.0 lands. Unpinned modules are how prod changes when nobody touched it.”

10 validation and testing
“tflint for mistakes the provider would reject, checkov for security findings, `terraform test` for behaviour. Clean, or every exception written down in an ADR with a reason.”

Close
“Contract, composition, stable keys, and refactors that destroy nothing. By Saturday dev and prod both deploy from the same modules, and Alpha is a real project.”